

SENTINEL - Full Review

Bugs, UI enhancements, integrations, and a next-level roadmap

Davis Delivery | reviewed at HEAD e98f3b1 (main) | generated 2026-06-24

This review covers the current state of the repo after the empty-KPI fix (#23), the dual-source travel-time + Anomalies tab feature (#24), and the expected-drive reference fix (#25). Two parallel specialist audits (backend, frontend/UX) read every file in scope; the most consequential claims were independently re-verified at runtime before inclusion here.

[CRITICAL] Headline finding

The two worst bugs found are in the dual-source travel-time code shipped in PR #24 -- the newest feature, already live and already backfilled across 6 months of records. One of them (C1) can produce false theft accusations. Both are detailed below and should be fixed before the numbers in this app are used in any HR conversation.

1. Bugs -- ranked findings

CRITICAL

[CRITICAL] C1 -- Motive per-leg drive math can fabricate theft (and hide it)

`_sentinel-scan.js:330-339, 457-463`. Two compounding defects, verified in the code:

1) A drive period only counts if it STARTS inside the leg window, and then counts in full -- no clipping to the window. The window anchors are NuVizz confirmation timestamps, and drivers often confirm a stop after already driving away. A real 72-minute return drive that started 4 minutes before the "last delivery" timestamp is excluded entirely.

`pickLeg` then accepts ANY positive period count -- even a 2-minute yard shuffle -- as "Motive actual." Result: expected travel = 2 minutes instead of Google's 40, and a legitimate day scores a critical gap with stolen dollars attached. The reverse also holds: an off-route joyride's drive time gets credited as expected travel, shrinking real theft.

This is live in production and in the records already backfilled across 2026. Some portion of the \$9.6k/30-day and \$75k/2026 totals is contaminated in both directions.

Fix: clip periods using their end time (already available), exclude off_route-classified drives from the leg credit, and treat a Motive sum below a sanity floor (e.g. under 25% of Google's estimate) as no-data so it falls back to Google. Then re-scan the affected window.

[HIGH] H1 -- Wrong-truck GPS still charges the wrong driver

The `routeMatch` reliability gate ("this GPS is not his route -- do not trust it") is applied to the travel-time legs but NOT to the in-route Class-3 scoring right below it (`_sentinel-scan.js:511+`). If someone else drove the truck that day, every stop they made classifies as off-route for THIS driver -> up to +40 risk score and stolen dollars charged to the wrong person, and their GPS pauses show up in the Anomalies tab under his name. The `motive_route_mismatch` data-health note gets written, but it does not stop the dollars or the score.

!`routeMatch`, skip `onwhere` out the in-route flag, score contribution, and stolen-minutes add-on; keep pauses/visits out of the Anomalies feed for that day (or mark them untrusted).

HIGH

ID	Finding	Where	Impact
----	---------	-------	--------

ID	Finding	Where	Impact
H2	Daily Audit counts no-data days as "clean" on the distribution bar and grouping -- the exact misleading-green problem already fixed in Investigate.	index.html:2126-2151	A falsely green audit page in a tool whose output feeds HR conversations.
H3	Anomalies row layout: the duration ("1h 40m") renders on top of the driver name on every row.	index.html: .off-row grid 24px 1fr auto reused for anomaly rows	Cosmetic, but on the newest, most-demoed screen.
H4	Investigate heatmap becomes unreadable at full history -- one flex square per workday; at ~350 days each square is a 1px sliver with overprinted text.	index.html heatmap render + CSS	Broken at exactly the data volume just backfilled.
H5	Nightly scan actually targets T-8, not T-7. Verified at runtime: <code>addDays("2026-07-07", -7)</code> returns "2026-06-29"; even <code>n=0</code> shifts a day back (UTC-midnight formatted in ET).	sentinel-nightly-scan.js <code>addDays()</code>	Detection runs a day later than designed. The backfill file already contains the FIXED version of this exact function -- just needs copying over.

MEDIUM (the ones that matter)

- `_distance.js` cost cliff: the rewrite changed cache keys to include a time bucket, orphaning the entire pre-rewrite cache -- the whole route matrix re-bills Google on the next full pass. Traffic-aware calls also bill at Google's Advanced Distance Matrix rate. And known-bad (non-routable) addresses are cached but never read back on lookup, so they re-bill on every single scan, forever.
- Anomalies "repeat places" fabricates patterns for null-ZIP pauses: every pause whose Motive address did not parse a ZIP groups into one bucket per driver, so two unrelated locations can register as a false "habitual spot" (James Bennett's "ZIP ?" pattern in the earlier run may be exactly this).
- The `anomalies` endpoint pulls the full `motive` map (pauses, visits, zips) for every record ever written (~15k docs), then filters by date in memory. Works today; degrades linearly as the corpus grows. Needs a date-bounded Firestore query or a rollout computed at scan time.
- Reference Metrics "expected drive" now mixes Motive-actual and Google-typical minutes into one median -- not a like-for-like comparison, and it also feeds into the driver baselines. Should use the stored Google-typical figure specifically for that column.
- Frontend races/leaks, all confirmed by direct code read: Daily Audit's Refresh button re-binds its date-change listener every time it is clicked (N taps -> N+1 duplicate fetches on every future date change); `navigateToDriver` double-binds the Investigate dropdown's change handler; Audit/Investigate/Anomalies have no "stale response" guard, so a slow older request can silently overwrite a newer view; Profiles re-binds its search listener after every maintenance action (Recompute Baselines / Rescore All).
- Records written by the pre-dual-source engine show a misleading "Motive actual: -- (no GPS match)" in the new Travel-times panel, implying a GPS lookup was attempted and failed, when none was ever attempted on that older record.
- `APP_VERSION` was not bumped across PRs #23-#25 even though the dashboard payload shape changed twice. It is the client cache key, so an old-shaped cached payload can still be served on a stale device. The shape guards make this graceful today, but it should be bumped on every shape change as a matter of process.

LOW (summarized)

Backfill reset has a narrow race that can strand the status doc mid-chain; the rescore evidence-note regex truncates detour text at the first decimal point ("via 3." instead of "via 3.4mi"); the raw-stops diagnostic sorts 12-hour timestamps as plain text, which orders them wrong; the dashboard's day-window math uses UTC "today" instead of Eastern, so the window silently shifts by one day after roughly 8pm ET; a dozen small unescaped-field and number-format nits across both files; the committed "How It Works" doc still describes the access secret that was deliberately removed; `user-scalable=no` in the page meta blocks pinch-zoom on mobile; and there is no automated test coverage beyond the three-case engine self-test.

2. UI / UX enhancement suggestions (prioritized)

- Deep-link everything to the exact driver-day. Tapping an anomaly row or a heatmap square should land on that specific day, auto-expanded -- today it just opens the driver's full history and leaves you to scroll. This is the single biggest friction point in the investigate workflow.
- Evidence export. The tool's entire purpose is an eventual HR conversation, and there is currently no way to export anything. Two cheap, client-side-only wins: a per-driver "Export CSV" of flagged days (date, shift, stops, gaps, flags, dollars), and a print stylesheet plus "Print evidence summary" button that formats one driver's flagged days as a clean one-page PDF via the browser print dialog.
- "Flagged only" filter and a worst-first sort on the Investigate daily breakdown -- five bad days should not be buried inside three hundred and fifty rows.
- Replace the day-per-square heatmap with a horizontally scrollable month grid, or a weekly-aggregate strip colored by that week's worst day. This properly fixes H4 instead of patching it, and stays legible at any tenure length on a phone screen.
- Cross-badge anomalies against theft flags: when the same driver-day has both a GPS anomaly and stolen dollars, badge it ("also \$42 flagged this day") so corroborated findings float above ordinary lunch stops.
- Session restore: persist the last-viewed tab, driver, audit date, and anomaly view in localStorage and restore them on load -- mobile browsers kill background tabs constantly and the app currently always reopens on a 30-day Overview.
- Daily Audit date picker: add previous/next-day arrows next to the date dropdown, show the weekday in each option label, and mark dates that contain a critical or high day directly in the list so problem days can be found by skimming rather than clicking through one at a time.
- Anomalies tab polish: a "last updated" timestamp with a manual refresh button, a driver name filter, and the cross-badge above.
- Accessibility quick wins: remove `user-scalable=no`; add visible keyboard focus outlines (currently suppressed globally); add `aria-pressed` to the range/view toggle chips.

3. Integrations worth adding

Integration	Why it matters
QuickBooks Payroll reconciliation	Already in use at Davis Delivery. Pull actual paid hours per pay period and show paid-vs-accounted per driver -- turns "stolen minutes" into "dollars actually paid out," which is the number that lands in a conversation with a driver or an owner. Highest-leverage integration by a wide margin.
Motive dashcam pull-through	A motive-dashcam endpoint already exists in this repo but nothing calls it yet. Wire each flagged Anomalies pause to a "View dashcam at 12:04pm" link -- a 117-minute unexplained stop with footage attached closes the case on the spot.
Scheduled digest notifications	A nightly summary via email, SMS, or Slack (Zapier already covers all three) -- "Yesterday: 2 critical days -- Bishop \$71, Head \$54" -- so the tool proactively surfaces problems instead of requiring someone to remember to open the dashboard.
NuVizz live API ingestion	Replacing the current weekly-email import is what would prevent the original empty-dashboard incident from recurring. A partial live-API tool already exists in the repo (nuvizz-loads-by-date.js) but targets endpoints that do not match NuVizz's actual API contract and needs real fixing before it is usable.
Reverse geocoding for anomaly locations	Turn a bare "ZIP 30180" into "residential address" or "gas station" via a geocoding lookup -- immediate, low-effort context for every repeated-place pattern the Anomalies tab surfaces.
Netlify site-level password	The app is fully public today by deliberate choice (the in-app secret was removed at the owner's request). This is the one-checkbox way to add a gate later without reintroducing any in-app secret, if that choice is ever revisited.

4. What takes this to the next level

1. Make the numbers defensible, first. Fix C1 and H1, re-scan the affected window, and add a per-day confidence grade based on data completeness (punch confirmed, deliveries confirmed, GPS present, route-match confirmed). An HR conversation needs "high-confidence: \$81, all three sources agree" -- not a number that might be a windowing artifact.
2. Case management. Turn flags into cases with a lifecycle -- open, reviewed, discussed, resolved -- plus operator notes and attached evidence per case. That converts the dashboard from a report into an investigation record, which doubles as the legal paper trail if it is ever needed.
3. Outcome tracking. The baselines and trend data already exist; add a simple before/after-confrontation view per driver to prove the tool is actually recovering money -- Montel Bishop alone, at roughly \$1,400 a month, would justify the entire project.
4. Tighten the detection loop. Run a lighter T-1 scan as soon as punches and GPS are available, with the current T-7 pass reserved for the full NuVizz reconciliation once that weekly data lands -- instead of everything waiting a full week end to end.
5. Engineering health, lower priority but worth planning for: a real test suite around the scoring math (today there are three self-test cases total), scan-time anomaly rollups so the Anomalies endpoint stays fast at ten times today's data volume, and eventually splitting the roughly three-thousand-line single-file frontend into modules.

Recommended sequencing

C1, H1, and H5 belong in a single focused pull request: clip and floor the Motive per-leg sums, gate the in-route (Phase B) scoring on `routeMatch`, and copy the already-fixed `addDays` function from the backfill file into the nightly-scan trigger. Re-scan the affected window once that lands. Everything else in this report can queue behind that fix.

Prepared as a companion to AUDIT.md and the SENTINEL "How It Works" reference document. Line numbers and function names reflect the repository at HEAD e98f3b1 on the main branch; verify against current source before acting on any single citation, as the codebase continues to change.